

Uno: a composable framework for nonlinearly constrained optimization

Charlie Vanaret¹ and Alexis Montoisson²

¹ Applied Optimization Department, Zuse-Institut Berlin, Germany ² Mathematics and Computer Science Division, Argonne National Laboratory, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Uno is a composable software framework for nonlinearly constrained optimization written in modern C++. It unifies the workflows of Lagrange-Newton methods, i.e., gradient-based algorithms that iteratively solve the KKT optimality conditions using Newton's method. As of April 2026, Uno supports sequential (convex and nonconvex) quadratic programming, interior-point (barrier) methods, sequential linear programming, and unconstrained optimization.

Uno breaks down optimization algorithms into reusable modular components such as step computation, constraint reformulation, globalization techniques, and acceptance criteria. This allows classical and hybrid methods to be configured and compared within a single framework. For full mathematical details of the algorithms implemented in Uno, see ([Vanaret & Leyffer, 2026](#)).

The core C++ code of Uno is organized into modular, object-oriented components that separate the mathematical logic of the algorithms from implementation details such as memory management, data structures, and computational routines. Uno has interfaces to Julia, Python, C, Fortran, and AMPL, enabling interoperability across scientific computing environments. Precompiled artifacts are available on GitHub, and the solver can be accessed directly via `UnoSolver.jl` in Julia or `unopy` in Python.

Statement of need

Nonlinearly constrained optimization is central to engineering, optimal control, machine learning, and scientific modeling ([Nocedal & Wright, 2006](#)). It also plays a key role in mixed-integer nonlinear optimization ([Lee & Leyffer, 2011](#)).

Typical nonlinear solvers share common building blocks such as step computation, constraint handling, and globalization strategies. These components can be found across major paradigms such as sequential quadratic programming, interior-point methods, and augmented Lagrangian methods, but these are usually developed as separate solver families. Algorithmic ideas are typically tested at the level of complete solvers, rather than individual components.

Most established open-source nonlinear optimization solvers are legacy codes, often written in Fortran or C, and tend to evolve slowly. This can make incorporating new algorithmic ideas or adapting to novel problem structures challenging. A notable exception is MadNLP.jl ([Shin et al., 2024](#)), a solver written in Julia and actively developed, though it remains focused on interior-point methods.

These observations motivate the development of a modern, composable framework in which algorithmic components are made explicit.

State of the field

Popular solvers such as IPOPT (Wächter & Biegler, 2006), KNITRO (Byrd et al., 2006), SNOPT (Gill et al., 2005), and WORHP (Büskens & Wassel, 2012) are robust and efficient, but they are typically monolithic: parameter tuning is exposed while internal components remain rigid. These solvers are primarily designed for end users rather than algorithmic experimentation. Testing new algorithmic variants usually requires intrusive modification or reimplementa-

Unification framework

Uno enables experimentation at the component level by providing a composable optimization framework in which algorithms are constructed from modular components that reflect mathematical concepts such as step computation, constraint reformulation, and globalization techniques.

Within this framework, strategies are organized into a coherent hierarchy, illustrated in the wheel of strategies (Figure 1): the outer ring represents high-level layers, the middle ring represents algorithmic ingredients that are combined automatically, and the inner ring lists possible strategies for each ingredient. The strategies currently implemented in Uno are highlighted in green.

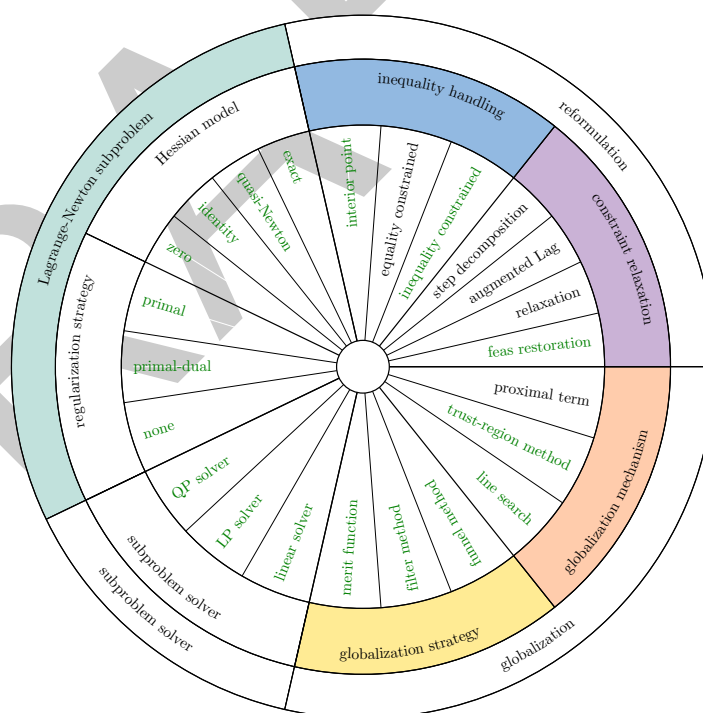


Figure 1: Unification framework: wheel of strategies.

Software design

In Uno's object-oriented architecture, the ingredients of Figure 1 are abstract classes whose interfaces should be implemented by subclasses (the strategies). Uno's simplified UML diagram is shown in Figure 2. Inheritance is represented as dashed lines with white arrows, while composition is represented as solid lines with black diamonds. Abstract classes are written in

60 italic, while subclasses are written in bold. This modular architecture offers a clear separation
61 between the mathematical logic of the optimization algorithm (the reformulation of the problem,
62 the definition of the subproblem, and the globalization techniques) and the computational
63 aspects (evaluating the model's functions, and solving the subproblems).

64 Uno implements a generic Lagrange-Newton method in which the abstract classes interact with
65 one another and exchange data, while being agnostic of the underlying strategies. Strategies
66 are picked by the user at runtime via options. Uno also implements presets, that is particular
67 combinations of strategies (and sets of hyperparameters) that correspond to state-of-the-art
68 solvers. Uno currently implements an *ipopt* preset that mimics the IPOPT solver (Wächter
69 & Biegler, 2006) (a *line-search restoration filter interior-point method with exact Hessian
70 and primal-dual inertia correction*), and a *filtersqp* preset that mimics the filterSQP solver
71 (Fletcher & Leyffer, 1998) (a *trust-region restoration filter SQP method with exact Hessian
72 and no inertia correction*). While, in theory, all combinations of strategies can be generated,
73 some are not supported yet (e.g., interior-point with a trust-region constraint).

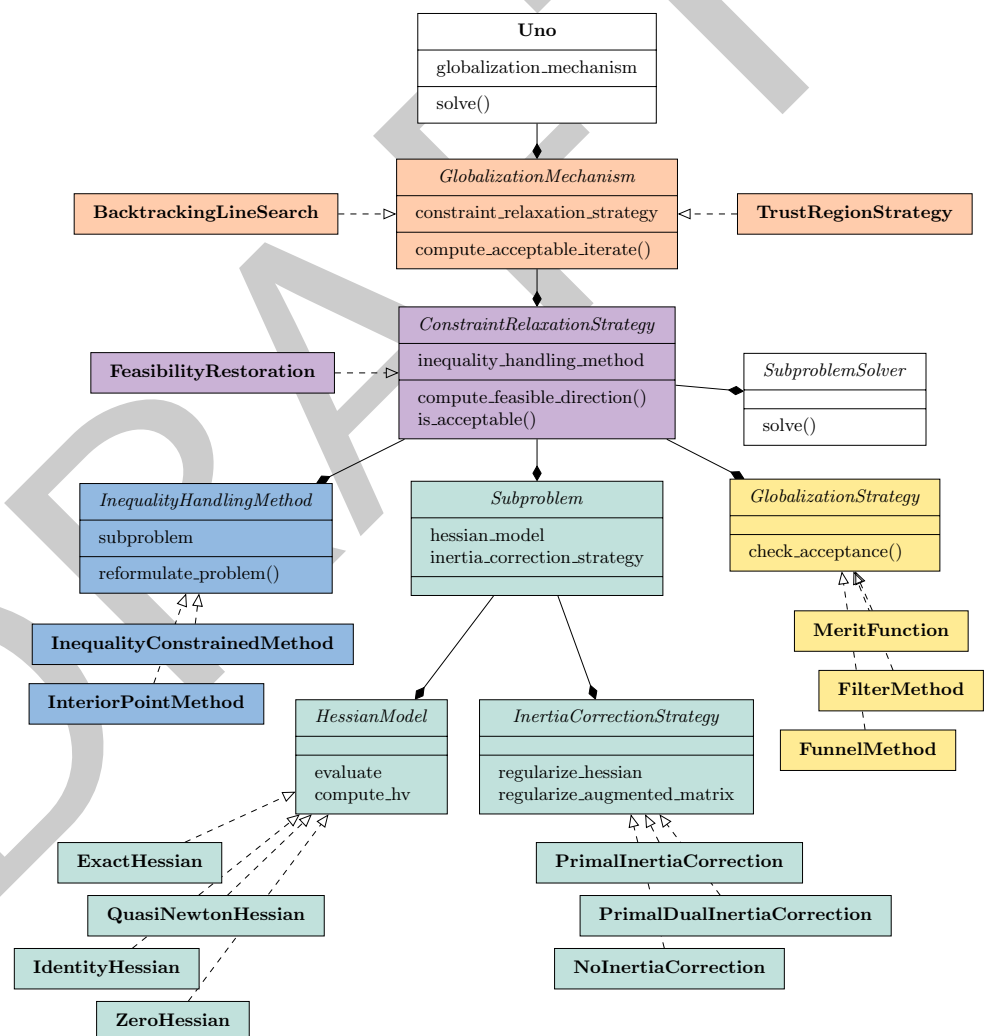


Figure 2: Uno's UML diagram.

74 Subproblem solvers are treated as interchangeable components: they can be plugged in or
75 swapped out without modifications of the algorithmic logic. This allows users and developers
76 to select the most appropriate solver for their problem structure, licensing constraints, or

77 performance requirements. Uno currently provides interfaces to several established LP, QP and
 78 linear solvers: BQPD (Fletcher, 2000), HiGHS (Huangfu & Hall, 2018), MUMPS (Amestoy et
 79 al., 2000), MA27 (I. S. Duff & Reid, 1983), MA57 (Iain S. Duff, 2004), SSIDS (Hogg et al.,
 80 2016), and Krylov.jl [montoison2023krylov].

81 Interfaces

82 To make Uno accessible to a wide range of users, we provide multiple language interfaces.

83 The AMPL Solver Library (Gay, 1997) interface gives access to the AMPL (Fourer et al., 1990)
 84 modeling language for optimization. It is distributed as a binary that takes a compiled AMPL
 85 model (.nl file) as input, allowing users to solve problems without directly interacting with the
 86 C++ core.

87 The C interface provides direct access to Uno's core functionality while maximizing
 88 interoperability with other programming languages and tools. The optimization process is
 89 expressed as the interaction between two independent opaque objects: a model describing the
 90 optimization problem and a solver holding the algorithmic configuration and execution state.
 91 The interface is therefore centered around two main structures:

- 92 ■ **Model:** represents an optimization problem and stores information about variables,
 93 bounds, constraints, the objective function, and derivative information. Users create
 94 a model and set its components (objective, constraints, derivatives, initial primal-dual
 95 point).
- 96 ■ **Solver:** represents the algorithm used to solve a given model. Users set solver options,
 97 attach callbacks, and access results such as primal and dual solutions, residuals, iteration
 98 counts, and performance metrics.

99 The Fortran interface provides access to Uno through the C API using `iso_c_binding`. It
 100 is split into two files: `uno_c.f90` for low-level C bindings, and `uno_fortran.f90` for Fortran-
 101 friendly wrappers that handle string arguments. The interface can be included directly in a
 102 source file or wrapped in a module for cleaner use statements. It is provided as source rather
 103 than a precompiled Fortran module (.mod) to maximize portability and interoperability across
 104 compilers and platforms.

105 The Julia interface is distributed as the registered package `UnoSolver.jl`. It integrates Uno
 106 into the Julia optimization ecosystem through:

- 107 ■ a thin wrapper around the full C API,
- 108 ■ an interface to `NLPModels.jl` for solving problems following the NLPModels API, such
 109 as `ADNLPModels.jl` or `ExaModels.jl`,
- 110 ■ an interface to `MathOptInterface.jl` for handling JuMP models.

111 Precompiled artifacts are downloaded automatically, making the package plug-and-play without
 112 requiring user compilation.

113 The Python interface is registered on PyPI as `unopy`, providing Uno bindings via precompiled
 114 wheels on most platforms.

115 A MATLAB interface is also under development, further expanding Uno's accessibility.

116 We plan to integrate Uno as a continuous relaxation solver within MINLP frameworks (e.g., SCIP
 117 (Bestuzheva et al., 2023)). This setting introduces the additional challenge of reoptimization,
 118 where consecutive solves share structural similarity and must be handled efficiently by reusing
 119 internal solver state, such as active sets, factorizations, and preallocated workspace, rather
 120 than restarting from scratch at each node.

Research impact statement

Uno's `ipopt` and `filtersqp` presets currently perform on a par with the state-of-the-art solvers IPOPT (Uno is slightly less robust) and filterSQP (Uno is slightly more robust) in terms of function evaluations on a set of 429 small CUTE instances (Bongartz et al., 1995). An up-to-date performance profile is maintained on Uno's GitHub README page.

Uno's ongoing developments were presented at several international conferences over the years (ISMP 2018, SIAM OP 2021, ICCOPT 2022, ISMP 2024, and ICCOPT 2025), and were the subject of invited talks at Zuse-Institut Berlin (2020), Argonne National Laboratory (2022), and KU Leuven (2025). This resulted in scientific cooperations with the MECO team at KU Leuven (Kiessling et al., 2024, 2025) and the HiGHS team in Edinburgh.

Uno is currently used as a nonlinear optimization solver in:

- Julia:
 - the JuMP.jl ecosystem,
 - control-toolbox, a collection of Julia packages for mathematical control and its applications,
- C++:
 - CasADi, an open-source tool for nonlinear optimization and algorithmic differentiation,
- Python:
 - pyOptSparse, an object-oriented framework for formulating and solving nonlinear constrained optimization problems in an efficient, reusable, and portable manner,
 - DNLP, an extension of CVXPY to general nonlinear programming,
- Fortran:
 - CUTEst, the Constrained and Unconstrained Testing Environment with safe threads for optimization software,
 - IMPL © /IMPL-DATA © by Industrial Algorithms Limited, a modeling and solving platform used in the process industries especially suited for economic, efficiency and emissions optimization and estimation.

Ongoing discussions and community interest indicate potential future integrations in CasADi, Pyomo, Minotaur, and the NEOS Server.

Acknowledgments

This work was supported by the Applied Mathematics activity of the U.S. Department of Energy Office of Science, Advanced Scientific Computing Research, under Contract No. DE-AC02-06CH11357. This work was also supported in part by NSF CSSI Grant No. 2104068.

AI usage disclosure

No generative AI was used in the creation of the software, interfaces, implementation of algorithms, or documentation. ChatGPT was used to check spelling, grammar, and clarity of the English text in this paper.

References

- Amestoy, P. R., Duff, I. S., L'Excellent, J.-Y., & Koster, J. (2000). MUMPS: A general purpose distributed memory sparse solver. *International Workshop on Applied Parallel Computing*, 121–130. https://doi.org/10.1007/3-540-70734-4_16

- Bestuzheva, K., Besançon, M., Chen, W.-K., Chmiela, A., Donkiewicz, T., Van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., & others. (2023). Enabling research through the SCIP optimization suite 8.0. *ACM Transactions on Mathematical Software*, 49(2), 1–21. <https://doi.org/10.1145/3585516>
- Bongartz, I., Conn, A. R., Gould, N., & Toint, P. L. (1995). CUTE: Constrained and unconstrained testing environment. *ACM Transactions on Mathematical Software (TOMS)*, 21(1), 123–160. <https://doi.org/10.1145/200979.201043>
- Büskens, C., & Wassel, D. (2012). The ESA NLP solver WORHP. In *Modeling and optimization in space engineering* (pp. 85–110). Springer. https://doi.org/10.1007/978-1-4614-4469-5_4
- Byrd, R. H., Nocedal, J., & Waltz, R. A. (2006). KNITRO: An integrated package for nonlinear optimization. In *Large-scale nonlinear optimization* (pp. 35–59). Springer. https://doi.org/10.1007/0-387-30065-1_4
- Duff, Iain S. (2004). MA57—a code for the solution of sparse symmetric definite and indefinite systems. *ACM Transactions on Mathematical Software (TOMS)*, 30(2), 118–144. <https://doi.org/10.1145/992200.992202>
- Duff, I. S., & Reid, J. K. (1983). The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Softw.*, 9(3), 302–325. <https://doi.org/10.1145/356044.356047>
- Fletcher, R. (2000). Stable reduced Hessian updates for indefinite quadratic programming. *Mathematical Programming*, 87(2), 251–264. <https://doi.org/10.1007/s101070050113>
- Fletcher, R., & Leyffer, S. (1998). User manual for filterSQP. *Numerical Analysis Report NA/181, Department of Mathematics, University of Dundee, Dundee, Scotland*, 35.
- Fourer, R., Gay, D. M., & Kernighan, B. W. (1990). AMPL: A mathematical programming language. *Management Science*, 36(5), 519–554. https://doi.org/10.1007/978-3-642-83724-1_12
- Gay, D. M. (1997). *Hooking your solver to AMPL*. Technical Report 93-10, AT&T Bell Laboratories, Murray Hill, NJ, 1993, revised.
- Gill, P. E., Murray, W., & Saunders, M. A. (2005). SNOPT: An SQP algorithm for large-scale constrained optimization. *SIAM Review*, 47(1), 99–131. <https://doi.org/10.1137/S0036144504446096>
- Hogg, J. D., Ovtchinnikov, E., & Scott, J. A. (2016). A sparse symmetric indefinite direct solver for GPU architectures. *ACM Transactions on Mathematical Software (TOMS)*, 42(1), 1–25. <https://doi.org/10.1145/2756548>
- Huangfu, Q., & Hall, J. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142. <https://doi.org/10.1007/s12532-017-0130-5>
- Kiessling, D., Leyffer, S., & Vanaret, C. (2025). A unified funnel restoration SQP algorithm. *Mathematical Programming*, 1–45. <https://doi.org/10.1007/s10107-025-02284-3>
- Kiessling, D., Vanaret, C., Astudillo, A., Décré, W., & Swevers, J. (2024). An almost feasible sequential linear programming algorithm. *2024 European Control Conference (ECC)*, 2328–2335. <https://doi.org/10.23919/ECC64448.2024.10590864>
- Lee, J., & Leyffer, S. (2011). *Mixed integer nonlinear programming*. Springer Science & Business Media. <https://doi.org/10.1007/978-1-4614-1927-3>
- Nocedal, J., & Wright, S. J. (2006). *Numerical optimization*. Springer. <https://doi.org/10.1007/978-0-387-40065-5>
- Shin, S., Anitescu, M., & Pacaud, F. (2024). Accelerating optimal power flow with GPUs: SIMD

- 209 abstraction of nonlinear programs and condensed-space interior-point methods. *Electric*
210 *Power Systems Research*, 236, 110651. <https://doi.org/10.1016/j.epsr.2024.110651>
- 211 Vanaret, C., & Leyffer, S. (2026). Implementing a unified solver for nonlinearly constrained
212 optimization. *Mathematical Programming Computation*. [https://doi.org/10.1007/s12532-](https://doi.org/10.1007/s12532-026-00310-9)
213 [026-00310-9](https://doi.org/10.1007/s12532-026-00310-9)
- 214 Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter
215 line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*,
216 106(1), 25–57. <https://doi.org/10.1007/s10107-004-0559-y>

DRAFT